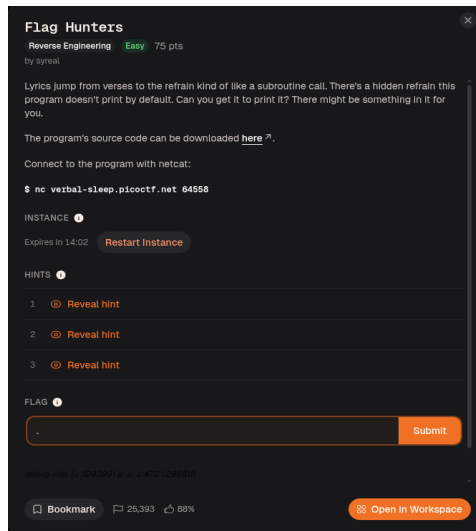


Flag Hunters - How I Found ;RETURN 0

Category	Reverse Engineering
Difficulty	Easy
Flag	picoCTF{70637h3r_f0r3v3r_c373964d}



1. First trial: I typed hello

The challenge gave a netcat connection. I connected and the program printed song lyrics. At the chorus it stopped and asked for Crowd:. I did not know the exploit yet, so I used a harmless test input: hello.

```
nc verbal-sleep.picoctf.net 64558
```

```
Crowd: hello
```

```
...
```

```
Crowd: hello
```

```
...
```

```
Crowd: hello
```

The important observation was that I typed hello only once, but Crowd: hello appeared again after every later chorus.

That told me the program was not simply printing my input once. It was saving my input inside the song data and reusing that modified song line later.

2. What the hello test told me

Because hello repeated without me typing it again, I suspected my input was being stored. The source confirmed it:

```
crowd = input('Crowd: ')
song_lines[lip] = 'Crowd: ' + crowd
```

So when I entered hello, the original CROWD line was replaced with Crowd: hello. That is why the same text kept appearing.

3. Then I found the command parser

The program splits every lyric line at a semicolon:

```
for line in song_lines[lip].split(';');
```

It also recognizes commands such as:

```
REFRAIN
RETURN 10
END
```

The source showed that RETURN number changes the lyric pointer (lip) to that line number:

```
elif re.match(r"RETURN [0-9]+", line):
    lip = int(line.split()[1])
```

4. Why I chose ;RETURN 0

The hidden intro containing the flag is placed at the beginning of the song, but the program normally starts from [VERSE1]. So I wanted to make the lyric pointer jump back to line 0.

The command for that is:

```
RETURN 0
```

But my input is first saved as text after Crowd:. I needed a semicolon so the parser would split my text and see RETURN 0 as a separate command. Therefore I tried:

```
;RETURN 0
```

The saved lyric line became:

```
Crowd: ;RETURN 0
```

On the next chorus, splitting at ; produced:

```
Crowd:
RETURN 0
```

Now the program treated RETURN 0 as a real instruction and jumped to the first lines of the song, where the secret intro and flag were stored.

5. Final test and result

```
Crowd: ;RETURN 0
```

```
Pico warriors rising, puzzles laid bare,
```

```
...
```

```
The ether's ours to conquer, picoCTF{70637h3r_f0r3v3r_c373964d}
```

Why did the flag repeat? The injected RETURN 0 stayed inside song_lines. Every later chorus parsed it again and jumped back to line 0. The loop stopped only because MAX_LINES = 100.

Simple takeaway

My first hello test showed that Crowd input was stored and reused. Reading the source then showed that semicolons split instructions and RETURN n controls the lyric pointer. So ;RETURN 0 turned my text input into an interpreter command that jumped to the hidden flag.

```
hello repeats -> input is stored -> ; splits commands -> RETURN 0 jumps to line 0 -> flag prints
```